SISTEMA DE AUTORIZACIÓN DE FACTURAS ELECTRÓNICAS CON SERVICIO WEB REST EN ASP.NET CORE Y APLICACIÓN WEB DE CONSULTA

202502375 – Erick Adrian Patzan Perez
Introducción a la Programación y Computación 2, sección D. Escuela de Ciencias y Sistemas, Facultad de Ingeniería, Universidad de San Carlos de Guatemala

Resumen

Este ensayo describe el diseño de un sistema que autoriza facturas electrónicas, un tema vigente en Guatemala y en el mundo por la expansión de la facturación digital y el control tributario. El sistema se divide en un servicio web REST, que valida y guarda la información, y una aplicación web que permite al usuario enviar archivos y consultar reportes. Se exponen las posturas adoptadas: separar responsabilidades por capas, elegir colecciones que reduzcan las búsquedas y emplear aritmética decimal para el cálculo del impuesto, además de mostrar los resultados con gráficas y reportes en PDF. Se analiza el impacto técnico, pues la validación automática evita errores manuales, y el impacto económico y social, ya que facilita la fiscalización y el cumplimiento de obligaciones. Entre las conclusiones se destaca que la separación entre servicio y pantalla simplifica las pruebas, que Dictionary y HashSet resuelven las búsquedas de forma eficiente y que decimal evita errores de redondeo en dinero.

Palabras clave: NIT, facturación electrónica, API REST, validación, ASP.NET Core

Abstract

This essay describes the design of a system that authorizes electronic invoices, a current topic in Guatemala and worldwide because of the growth of digital invoicing and tax control. The system is divided into a REST web service, which validates and stores the information, and a web application that lets the user send files and query reports. The adopted positions are presented: separating responsibilities into layers, choosing collections that reduce searches, and using decimal arithmetic to calculate the tax, besides showing the results with charts and PDF reports. The technical impact is analyzed, since automatic validation prevents manual errors, along with the economic and social impact, since it eases oversight and compliance with obligations. Among the conclusions, it is highlighted that separating the service from the user interface simplifies testing, that Dictionary and HashSet solve lookups efficiently, and that decimal avoids rounding errors with money.

Keywords: *TIN, electronic invoicing, REST API, validation, ASP.NET Core*

Introducción

La facturación electrónica obliga a las entidades tributarias a revisar grandes cantidades de documentos con rapidez y sin errores. Este ensayo presenta un sistema que recibe el registro de contribuyentes y las facturas en archivos JSON, comprueba cada una y asigna un número de autorización a las correctas. Se busca responder: ¿cómo organizar el programa para que sea fácil de probar y mantener?, ¿qué estructuras de datos conviene usar? y ¿cómo asegurar que el cálculo del impuesto sea exacto? El aporte para el lector es una explicación clara de las decisiones tomadas y de los algoritmos principales, con diagramas de apoyo en los apéndices.

Desarrollo del tema

La solución se plantea de forma deductiva: primero se presenta la arquitectura general y después los casos particulares, que son las estructuras de datos, los algoritmos de validación y las pruebas.

a. Arquitectura por capas

El sistema se divide en dos programas que se comunican por HTTP. El primero es un servicio web con estilo REST (Fielding, 2000), construido con ASP.NET Core (Microsoft, 2024a). El segundo es una aplicación de Razor Pages que solo muestra pantallas: toma lo que el usuario escribe o sube, lo envía al servicio con HttpClient y presenta la respuesta. Así, ninguna regla de negocio vive en la pantalla.

Dentro del servicio se separan los controladores, que reciben las peticiones; los servicios, que contienen la lógica; los modelos, que describen los datos; y las utilidades, que son funciones pequeñas como validar un NIT. Esta división sigue la idea de separar la lógica de negocio de la presentación (Fowler, 2002) y permitió probar cada parte por separado.

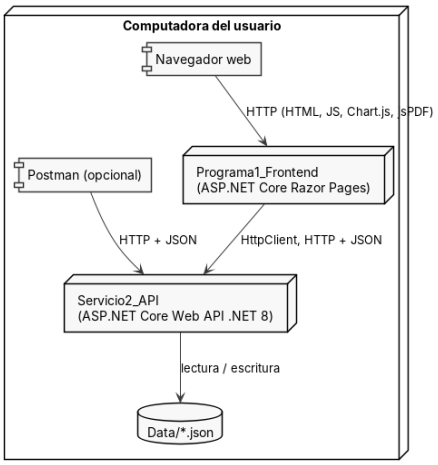


Figura 1. Diagrama de despliegue del sistema.
Fuente: elaboración propia.

b. Estructuras de datos

Se usaron únicamente las colecciones propias de C#, escogidas según la consulta más frecuente de cada dato. Las tablas hash ofrecen búsqueda e inserción en tiempo promedio constante (Cormen et al., 2009), por lo que Dictionary y HashSet evitan recorrer listas completas.

Tabla I.
Colecciones utilizadas y su propósito

COLECCIÓN	PROPÓSITO
Dictionary (NIT)	Buscar un contribuyente del RTU por su NIT sin repetirlo.
HashSet (referencia)	Detectar referencias duplicadas: Add devuelve falso si ya existía.
List (facturas)	Guardar todas las facturas recibidas en orden de llegada.
Dictionary (día)	Llevar el correlativo siguiente de cada fecha.

Fuente: elaboración propia.

c. Algoritmos de validación

El NIT debe tener de 1 a 20 dígitos y un dígito verificador, que puede ser un número o la letra K. Para comprobarlo se recorre el cuerpo del NIT de derecha a izquierda, multiplicando cada dígito por un peso que empieza en 2 y aumenta de uno en uno. El verificador esperado se obtiene con la ecuación (1); si el resultado es 10, se espera la letra K.

$$v = (11 - (S \bmod 11)) \bmod 11 \qquad (1)$$

donde:

v = dígito verificador esperado

S = suma de cada dígito por su peso

Con este método se comprobaron NIT de ejemplo: 7378106 y 8338817 resultan válidos, mientras que 5123456 no. Un NIT con formato correcto pero que no está en el RTU se cuenta aparte, como inexistente.

Para el impuesto se usó el tipo decimal en lugar de double, porque este último puede arrastrar errores de representación que son inaceptables con dinero (Microsoft, 2024b). El IVA correcto se calcula con la ecuación (2) y el total esperado con la (3), ambos redondeados a dos decimales.

$$IVA = redondear(V \times 0.12, 2) \qquad (2)$$

$$TOTAL = V + IVA \qquad (3)$$

donde V es el valor de la factura sin impuesto. Una factura se aprueba solo si no tiene ningún error; entonces recibe un número formado por la fecha de la solicitud (aaaammdd) y un correlativo de 8 dígitos que reinicia cada día. Los diagramas de actividad de estos procesos están en los apéndices.

d. Interfaz y reportes

La aplicación web ofrece cuatro grupos de opciones: enviar mensaje, inicializar, peticiones y ayuda. Al enviar un archivo, una clase llamada DetectorMensaje lo lee, decide si corresponde al registro de contribuyentes o a una solicitud de facturas, y muestra una vista previa con los primeros registros. Solo después de que el usuario confirma, el texto original se manda al servicio, sin modificarlo.

Las peticiones permiten consultar el reporte de facturas por fecha, el movimiento de un contribuyente emitido o recibido, el resumen de IVA de un día y el valor autorizado dentro de un rango de fechas, con o sin impuesto. Los resultados de IVA y de rango se dibujan como gráficas de barras en el navegador y cualquier consulta puede descargarse como PDF. Para que el sistema funcione sin conexión a internet, las librerías de gráficas y de PDF se incluyeron dentro del propio proyecto.

Tabla II.

Errores que cuenta el reporte por fecha

ERROR	CUÁNDO SE REGISTRA
NIT inválido	El formato o el dígito verificador no cumplen la ecuación (1).
NIT inexistente	El NIT es válido, pero no está en el registro de contribuyentes.
IVA mal calculado	El IVA recibido es distinto al de la ecuación (2).
TOTAL mal calculado	El total recibido es distinto al de la ecuación (3).
Referencia duplicada	La referencia ya llegó en una factura anterior.

Fuente: elaboración propia.

El reporte cuenta cada error por separado y también cuántas facturas quedaron sin errores, cuántos emisores y cuántos receptores distintos hubo en cada fecha. Esto permite ver de un vistazo qué día tuvo más problemas y de qué tipo.

e. Persistencia de los datos

El servicio guarda el registro de contribuyentes, las facturas, las referencias y los correlativos en archivos JSON. Cada escritura se hace primero en un archivo temporal y luego se reemplaza el original, de modo que un cierre inesperado no deja un archivo a medias. Al arrancar, el servicio carga esos archivos, por lo

que el siguiente número de autorización continúa donde se quedó y una referencia ya recibida sigue considerándose duplicada. La opción Inicializar borra todo y reinicia los contadores.

f. Pruebas y decisiones

Se realizaron 35 verificaciones automáticas sobre los nueve endpoints del servicio y 45 sobre las pantallas, usando un navegador automatizado. Algunas reglas del enunciado admitían más de una lectura, así que se tomaron estas decisiones y se dejaron documentadas:

- El TOTAL se compara contra VALOR más el IVA recibido, con una constante para cambiarlo.
- Una factura con fecha ilegible se ignora y queda un aviso en el registro.
- El NIT inválido y el inexistente se reportan en columnas distintas.
- El correlativo usa la fecha de la solicitud y no la de la factura.

Conclusiones

- Dividir el sistema en un servicio y una pantalla permitió probar las reglas de negocio sin depender de la interfaz.
- Dictionary y HashSet resolvieron las búsquedas de NIT y referencias sin recorrer listas, lo que conviene cuando crece el volumen de facturas.
- El tipo decimal fue necesario para que el IVA y el redondeo dieran siempre el valor esperado.
- Guardar los datos en archivos permitió que el correlativo y las referencias se mantuvieran entre ejecuciones.

Como reflexión queda una pregunta abierta: ¿qué cambiaría si el servicio tuviera que autorizar miles de facturas por segundo? Una base de datos y procesamiento en paralelo serían el siguiente tema a investigar.

Referencias bibliográficas

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to algorithms (3.ª ed.). MIT Press.

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Tesis doctoral, University of California, Irvine].

Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.

Microsoft. (2024a). ASP.NET Core documentation. https://learn.microsoft.com/aspnet/core

Microsoft. (2024b). Decimal struct. .NET API reference. https://learn.microsoft.com/dotnet/api/system.decimal